

Informe de Función Heurística

Proyecto 2 - Knight Energy

Integrantes

Samuel Arenas Valencia	202341928-3743
María Juliana Saavedra	202344035-3743
Libardo Alejandro Quintero	202342181-3743
Juan David Rincón	202332032-3743

Inteligencia Artificial
Universidad del Valle
2026-I

1. Contexto del problema

Knight Energy es un juego adversarial entre dos caballos sobre un tablero de ajedrez. La máquina juega con el caballo blanco (*white*) y el jugador humano con el caballo negro (*black*). En la interfaz del juego, esta representación lógica se acompaña con una identidad visual: el caballo del jugador se muestra dorado y el caballo controlado por la inteligencia artificial se muestra con apariencia plateada, manteniendo internamente la distinción blanco/negro para el cálculo de turnos, puntos y movimientos.

Cada caballo se desplaza siguiendo el movimiento en forma de “L” propio del ajedrez, consume energía al moverse y puede recolectar elementos del tablero que modifican el estado de la partida. Las estrellas aumentan la puntuación y las pociones permiten recuperar energía, por lo que una buena decisión no depende únicamente del puntaje actual, sino también de las opciones futuras que quedan disponibles después de cada movimiento.

El objetivo de la inteligencia artificial es elegir movimientos que aumenten sus posibilidades de terminar la partida con más puntos que el rival. Para lograrlo, el proyecto utiliza minimax con poda alfa-beta y una profundidad limitada según la dificultad seleccionada. Debido a que no siempre es posible explorar todo el árbol de juego hasta un estado terminal, se define una función heurística capaz de estimar qué tan favorable es un estado intermedio para la máquina.

Idea central

El algoritmo minimax necesita una forma de valorar los estados del juego. Cuando el estado es final, se usa una función de utilidad. Cuando el estado no es final pero se alcanzó la profundidad límite del árbol, se usa una función heurística ponderada.

2. Repositorio del proyecto

El código fuente completo de Knight Energy, incluyendo la implementación del estado del juego, la interfaz, el algoritmo minimax y la función heurística, se encuentra disponible en el siguiente repositorio:

[samuelArenas2005/Proyecto_IA_II](https://github.com/samuelArenas2005/Proyecto_IA_II)

3. Función de utilidad

La función de utilidad evalúa estados terminales, es decir, posiciones en las que la partida ya terminó. Como la inteligencia artificial corresponde al caballo blanco, los valores positivos favorecen a la máquina y los valores negativos favorecen al jugador.

$$U(s) = \begin{cases} 10000 + (P_w - P_b), & \text{si gana white} \\ -10000 + (P_w - P_b), & \text{si gana black} \\ 0, & \text{si hay empate} \end{cases}$$

Donde:

- P_w representa los puntos de la IA.

- P_b representa los puntos del jugador.
- $P_w - P_b$ representa la diferencia de puntos.

El valor 10000 se usa para que ganar o perder tenga más importancia que cualquier ventaja parcial. Así, minimax prefiere una victoria real sobre una posición intermedia aparentemente favorable.

4. Función heurística ponderada

El árbol minimax tiene profundidad limitada según la dificultad seleccionada: profundidad 2 para principiante, profundidad 4 para amateur y profundidad 6 para experto. Por eso, muchas veces el algoritmo no alcanza a llegar hasta un estado terminal. En esos casos, se necesita estimar qué tan bueno es un estado intermedio.

La heurística implementada es:

$$H(s) = 100(P_w - P_b) + 10(E_w - E_b) + 5(M_w - M_b) + 20(A_w^* - A_b^*) + 8(A_w^e - A_b^e) + B(s)$$

Donde:

- $E_w - E_b$ es la diferencia de energía.
- $M_w - M_b$ es la diferencia de movilidad, medida como cantidad de movimientos legales disponibles.
- $A_w^* - A_b^*$ es la diferencia entre el valor de estrellas alcanzables en el siguiente movimiento.
- $A_w^e - A_b^e$ es la diferencia entre el valor de casillas de energía alcanzables en el siguiente movimiento.
- $B(s)$ es la bonificación o penalización por bloqueo.

5. Penalización por bloqueo

La regla del juego indica que si un jugador no tiene energía suficiente para moverse, pierde el turno y se le descuentan 3 puntos. Por esta razón, la heurística también debe anticipar estados donde un jugador queda sin movimientos efectivos.

$$B(s) = \begin{cases} -50, & \text{si white queda bloqueado} \\ +50, & \text{si black queda bloqueado} \\ 0, & \text{si ninguno queda bloqueado} \end{cases}$$

Importancia estratégica

Si la IA queda bloqueada, el estado se considera peligroso y el valor heurístico disminuye. Si el jugador queda bloqueado, el estado se considera favorable y el valor heurístico aumenta.

6. Justificación de los pesos

Componente	Peso	Justificación
Diferencia de puntos	100	Es el factor más importante porque el ganador se decide por puntos.
Diferencia de energía	10	La energía permite seguir moviéndose y evita perder turnos.
Diferencia de movilidad	5	Más movimientos disponibles significan más opciones estratégicas.
Estrellas alcanzables	20	Las estrellas representan oportunidades inmediatas de obtener puntos.
Energía alcanzable	8	La energía ayuda a sostener la partida, pero no da puntos directamente.
Bloqueo	50	Anticipa el riesgo de perder turnos y recibir penalizaciones.

7. Fragmento de implementación

El siguiente fragmento resume la parte final de la función heurística y muestra cómo se combinan los componentes ponderados. No corresponde a la función completa, ya que en la implementación real también se calculan previamente valores auxiliares como `star_options_diff`, `energy_options_diff` y `blocked_score`. En particular, el código completo incluye el cálculo de las estrellas alcanzables por cada jugador, las casillas de energía alcanzables y la penalización o recompensa asociada a que un jugador quede bloqueado sin movimientos efectivos.

```

1 def heuristica_utility_function(self):
2     if self.is_end_game():
3         return self.utility_function()
4
5     white_moves = self.get_valid_moves("white") if self.
6         can_player_move("white") else []
7     black_moves = self.get_valid_moves("black") if self.
8         can_player_move("black") else []
9
10    points_diff = self.white_points - self.black_points
11    energy_diff = self.white_energy - self.black_energy
12    mobility_diff = len(white_moves) - len(black_moves)
13
14    return (
15        100 * points_diff
16        + 10 * energy_diff
17        + 5 * mobility_diff
18        + 20 * star_options_diff
19        + 8 * energy_options_diff
20        + blocked_score
21    )

```

8. Integración con minimax

La función heurística se usa cuando minimax alcanza la profundidad máxima y el juego todavía no ha terminado. En cambio, si el estado ya es terminal, se usa la función de utilidad.

Criterio de decisión

En los turnos de *white*, minimax maximiza el valor del estado. En los turnos de *black*, minimax minimiza el valor del estado. De esta forma, la máquina asume que el jugador intentará responder con el movimiento más perjudicial para la IA.

9. Conclusión

La heurística ponderada permite que la IA tome decisiones más estratégicas en estados no terminales. La función no solo considera los puntos actuales, sino también la energía, la movilidad, las oportunidades inmediatas de capturar estrellas o recuperar energía, y el riesgo de quedar bloqueado. Esto produce una evaluación más completa del estado del juego y mejora la calidad de las decisiones tomadas por minimax.

Además, la separación entre la representación visual y la representación lógica del juego facilita la implementación del algoritmo. Aunque en pantalla el jugador se identifica con el caballo dorado y la IA con el caballo plateado, internamente se conservan los roles *black* y *white*. Esta decisión permite que las funciones de utilidad, heurística y generación de movimientos trabajen con una estructura clara y consistente.

En conjunto, la función heurística actúa como una aproximación práctica al valor real de una posición cuando la búsqueda no puede llegar hasta el final de la partida. Gracias a esta evaluación, la IA puede priorizar movimientos que no solo generan beneficios inmediatos, sino que también conservan energía, mejoran su movilidad y reducen el riesgo de quedar en desventaja frente al jugador.

10. Añadido: heurísticas configurables y torneo

Como extensión del proyecto, se añadió la posibilidad de trabajar con diferentes funciones heurísticas. Cada heurística puede tener un nombre propio y una combinación distinta de pesos para los parámetros evaluados: diferencia de puntos, diferencia de energía, movilidad, estrellas alcanzables, energía alcanzable y bloqueo. Esto permite comparar estilos de juego diferentes, por ejemplo heurísticas más agresivas, defensivas, centradas en recolectar puntos o enfocadas en conservar energía.

También se incorporó un nuevo parámetro llamado control central. Este valor mide qué tan cerca se encuentra cada caballo del centro del tablero. La idea es que un caballo ubicado en posiciones centrales suele tener más posibilidades de movimiento que uno ubicado en bordes o esquinas. Por eso, el control central se calcula como una diferencia entre la centralidad del caballo *white* y la centralidad del caballo *black*. Si la IA está mejor posicionada en el centro, este componente aumenta la evaluación; si el rival tiene mejor control central, la disminuye.

En los modos de comparación entre inteligencias artificiales, minimax conserva la misma estructura de búsqueda con poda alfa-beta, pero la evaluación de los estados intermedios se

realiza con la heurística seleccionada. De esta forma, dos IA pueden jugar con funciones de evaluación diferentes aunque respeten las mismas reglas del juego.

Finalmente, se añadió un torneo de heurísticas. En este modo se seleccionan 2, 4, 8 o 16 heurísticas y se enfrentan por rondas: final, semifinales, cuartos u octavos según la cantidad de participantes. Cada enfrentamiento directo se juega intercambiando roles para reducir la ventaja de color: una partida con una heurística como *white* y la otra como *black*, y luego una segunda partida con los roles invertidos. La ganadora se decide primero por partidas ganadas, luego por puntos acumulados y, si aún existe empate, mediante una partida adicional de desempate. Esto permite observar qué combinación de pesos produce mejores resultados frente a otras estrategias.